VeriFlow · Team VeriFlow · FinTechathon 2026 International Track · Topic E · SYNTHETIC sandbox data

VeriFlow — Security Self-Assessment

Scope: VeriFlow v1.0 competition prototype (FinTechathon 2026, Topic E). It runs on SYNTHETIC data, and every financial action is simulated in a sandbox.
Assessment type: self-assessment by the team. This is not an independent audit and not a compliance certification.

---

1. Security objectives

1. The AI agent can never move money, change a payment status, or redirect a payment.
2. Consequential actions happen only after explicit approval by an authorised human, and that human is never the proposer.
3. Uploaded data is treated as evidence, never as instructions.
4. Every material decision is attributable and its record is tamper-evident.
5. No financial figure is invented: every number traces to evidence or a stated computation.
6. Nobody can mistake a simulation for a real financial operation.

2. Permission tiers (implemented)

Tier	Who	Can	Cannot	Enforced in
Analyst	Finance/operations assistant	View overview, cases, evidence, cash outlook, financing, audit; ask the agent; propose a follow-up	Approve or reject actions	`actions._check_approver` → HTTP 403 + `approve_denied` / `reject_denied` audit entry
Owner	Merchant owner / approver	Everything an analyst can, plus approve or reject proposals	Approve or reject a proposal they created themselves	`config.APPROVER_ROLES`, proposer ≠ approver check
Agent (`veriflow-agent`)	AI agent	Read verified results; answer; propose **one** allowlisted action type	Approve or reject anything; execute; choose amount or target; change evidence	Agent identity is reserved (`veriflow-*` actor names are rejected from clients); the allowlist sits in `security.ACTION_ALLOWLIST`
System (`veriflow-engine`, `-guard`, `-sandbox`)	Internal services	Reconcile; flag untrusted text; execute an *approved* action in the sandbox	Execute anything that is not approved	`actions.approve` → `_execute` is the only execution path

Capability removal (defence in depth). These capabilities simply do not exist anywhere in the code base:

- mark a settlement paid;
- edit a bank account or beneficiary;
- transfer funds;
- submit a loan application;
- send an external message.

A successful prompt injection therefore has nothing dangerous to call.

Action state machine. `proposed → approved → executed | failed` or `proposed → rejected`. Decisions are accepted only in the `proposed` state, so a rejected action cannot be approved later (tested).

3. Untrusted-input boundary

Control	Implementation	Evidence
Classify free text as data	Free-text fields per dataset are listed in `load.TEXT_FIELDS`	—
Detect instruction-like text	5 pattern families: override rules, force settlement status, change payment destination, approval bypass, role hijack (`security.INSTRUCTION_PATTERNS`)	`DLV-023` note flagged with `override_rules` , `force_settlement_status` , `change_payment_destination`
Log and ignore	`untrusted_instruction_detected` audit entry with outcome `ignored` ; a security-notice case is shown in the UI	`logs/audit_export.json`
Keep it out of the LLM	Flagged text is replaced by `[WITHHELD …]` ; other notes are escaped, truncated and placed inside `<untrusted_evidence_text>` tags; the system prompt forbids following data instructions	`test_6b`
Truth comes only from evidence	Settlement status is computed only from bank receipts; free text is never read by the reconciliation engine	`test_6` — `SET-004` stays overdue
Agent refusal	A user request to "follow the note" is answered with a refusal, no action is proposed, and an `agent_refused_untrusted_instruction` entry is written	`test_6` , sandbox log step 6

4. Model-output controls

The output of the LLM is treated as untrusted too:

1. **Schema:** structured JSON output is required. A parse failure falls back to the deterministic engine.
2. **Evidence ids:** ids are filtered against the loaded evidence, and unknown ids are dropped and noted.
3. **Numeric grounding:** any money-like figure that is not a computed value replaces the whole answer with the deterministic explanation (`test_claude_output_is_guarded`).
4. **Prohibited claims:** fraud/theft language and claims of having marked, sent, paid, transferred or approved anything cause replacement (`test_claude_prohibited_claims_replaced`).
5. **Actions:** only an allowlisted type for an actionable case is accepted. Amount and target are copied from `settlements.csv` .

6. **Availability:** on a network or auth failure, or a model refusal, the agent falls back to the rule-based engine and the UI shows the engine used.

5. Data protection and integrity

- **Data:** synthetic only, with no real names, phone numbers, accounts or identity documents. The masked synthetic account appears as `****0001 (SYNTHETIC)`.
- **Source evidence:** read-only. No API writes to evidence. Malformed rows are excluded and disclosed, not repaired.
- **File hashes:** the SHA-256 of every evidence file is recorded in the `dataset_loaded` audit entry.
- **Audit log:** append-only table with a SHA-256 hash chain (`prev_hash` + canonical JSON of the entry). `/audit/verify` recomputes it, and the UI displays the chain status. Editing any row breaks verification (`test_audit_chain_detects_tampering`).
- **Secrets:** the API key comes only from the `ANTHROPIC_API_KEY` environment variable. It is never logged or stored, and it is never needed for the demo.
- **Data sent to the LLM:** only aggregated, verified results and escaped synthetic notes. No real personal data exists to leak.

6. Honest-status controls

- A draft is labelled `DRAFT - NOT SENT` in the text, in `draft_not_sent` status, and in every UI message.
- The follow-up is an internal sandbox case, never described as a filed courier dispute.
- Confirmed cash and expected inflows are shown separately and never summed into a headline number.
- Overdue receivables are never called fraud or loss (tested).
- Financing results are labelled "sample terms", "appears to fit — not a credit approval", with a disclaimer.
- A permanent banner reads: SYNTHETIC SANDBOX DATA.

7. Known risks and residual exposure

#	Risk	Likelihood (prototype)	Current mitigation	Residual / next step
R1	**Role spoofing:** roles come from the `X-Role` header, not authentication	High if exposed publicly	Demo is run locally; every decision is audited with actor and role	Add authenticated sessions (OIDC), server-side role store, MFA for approvers
R2	**Prompt injection that evades the regex detector**	Medium	Detection is only one layer: status derives from bank evidence, no dangerous capability exists, notes are escaped and tagged, model output is guarded	Add an LLM-based classifier and broader multilingual patterns; red-team corpus
R3	**False match**, e.g. a receipt with the right reference and amount but for a different reason	Low–Medium	Exact reference + amount, one-to-one, duplicates detected	Use bank transaction ids; ask the user to confirm ambiguous matches

#	Risk	Likelihood (prototype)	Current mitigation	Residual / next step
R4	**Missed match:** courier paid under a different reference	Medium	Reported as an unapplied receipt or an open case, never silently netted; listed as an uncertainty	Assisted matching with explicit "suggested match" status requiring approval
R5	**Stale or incomplete statements**	Medium	Fixed as-of date; limitations listed; malformed rows excluded and disclosed	Freshness timestamps per source; warn when evidence is older than N days
R6	**Wrong due-date assumption**	Low	Overdue only from an evidenced due date; otherwise pending or unresolved	Contract-term ingestion
R7	**Misleading forecast**	Medium	Conservative vs expected cases with explicit assumptions; obligations outside the file are disclosed as not covered	Delay what-if scenarios; confidence ranges from history
R8	**Over-trust in AI explanations**	Medium	Every answer cites ids; evidence drawer; uncertainties shown; numbers come from the engine	User education; per-sentence citations
R9	**Model hallucination of figures or claims**	Medium	Numeric grounding and prohibited-claim guards with deterministic replacement	Extend grounding to dates and ids in free text
R10	**Audit log deletion:** the chain detects edits but an attacker with database access could truncate it	Low	Head hash displayed and exported (`logs/audit_export.json`)	Periodically anchor the head hash externally (e.g. a WORM store)
R11	**Local database not encrypted**	Low (synthetic data)	No real data stored	Encrypt at rest; least-privilege file permissions
R12	**Denial of service / large uploads**	Low (no upload endpoint)	Only fixed local datasets; question length capped at 1,000 characters	Size limits and rate limiting when uploads are added
R13	**Third-party dependency risk**	Low	Pinned versions in `requirements.txt` ; licences listed in the README	Dependency scanning in CI
R14	**Regulatory misinterpretation:** financing seen as advice	Low	Informational labels, no eligibility claim, lowest-cost logic disclosed	Legal review before any real lender integration

8. Out of scope (deliberately not implemented)

- Real money movement, bank transfers or beneficiary changes.
- Loan applications or credit decisions.
- Sending real messages to couriers.
- Live bank, courier or marketplace connections.

- Production identity management.
- Fraud detection or attribution.

None of these is represented as working.

9. Verification evidence

- `python -m pytest -q` passes 23 tests (permissions, rejection, injection, guards, audit integrity).
- `logs/sandbox_run.log` shows, in order: injection detected and ignored → agent refusal → rejection with no side effects → analyst approval denied (HTTP 403) → owner approval → sandbox execution with the draft not sent → hash chain valid.
- `logs/audit_export.json` contains the complete audit trail with its chain verification.